Step 2 — The SQL Generator and the Answer — Solution

> Back to the step · Hint 1 · Hint 2

Story: this turns the user's plain-English question into a real `SELECT` statement using structured output, and phrases the resulting rows back into one readable sentence. **If not:** you'd be parsing SQL out of free-form model prose by hand, and the user would end up seeing raw database tuples instead of an actual answer.

The query shape

```
# src/schemas.py
# The structured shape the model fills in: a single SQL query string.
from pydantic import BaseModel, Field


class SQLQuery(BaseModel):
    sql: str = Field(
        description="A single read-only SQL SELECT query for SQLite."
    )
```

The SQL generator (structured output)

```
# src/sql_agent.py
# Turn a plain-English question into a single read-only SELECT, using the
# schema so the model only uses real tables and columns. Structured output
# gives a clean SQL string, not prose.
from langchain_core.prompts import ChatPromptTemplate

from src.schemas import SQLQuery

SQL_PROMPT = (
    "You write ONE read-only SQL SELECT query for SQLite that answers the "
    "question, using ONLY the tables and columns in the schema below. Do "
    "not write INSERT, UPDATE, DELETE, DROP, or more than one statement.\n\n"
    "Schema:\n{schema}\n\nQuestion: {question}"
```

```
)


def generate_sql(model, schema, question):
    prompt = ChatPromptTemplate.from_template(SQL_PROMPT)
    chain = prompt | model.with_structured_output(SQLQuery)
    result = chain.invoke({"schema": schema, "question": question})
    return result.sql
```

The grounded answer

```
# src/answer.py
# Phrase the query result in one plain sentence, using ONLY the rows - so
# the answer reflects the data, not the model's guesses.
from langchain_core.prompts import ChatPromptTemplate

ANSWER_PROMPT = (
    "Answer the question in one plain sentence using ONLY the query result "
    "rows below. If the rows are empty, say no matching data was found.\n\n"
    "Question: {question}\nRows: {rows}"
)


def summarize_answer(model, question, rows):
    prompt = ChatPromptTemplate.from_template(ANSWER_PROMPT)
    chain = prompt | model
    reply = chain.invoke({"question": question, "rows": str(rows)})
    return reply.content
```

Why this split

- **`with_structured_output(SQLQuery)`** is what makes the SQL usable: you get `result.sql` — a clean string to guard and run — instead of a sentence like "Sure, here's a query: ...". Same tool as Doc05's structured output.
- **The schema in the prompt** keeps the model honest: it can only reference tables and columns that actually exist, which cuts down on queries that fail.
- **The answer step is grounded in the rows.** "Using ONLY the rows" is the same discipline as every RAG answer — the model turns `[(3,)]` into "There are 3 completed orders", and says "no matching data" on an empty result instead of inventing a number.

Tests for this step: proven live in Step 3. For now, with a model, `generate_sql` returns a string that starts with `SELECT`, and `summarize_answer(model, q, [(3,)])` returns a non-empty string.

The full behavior — question to guarded query to answer — is the integration test in Step 3.